Especificación de requisitos de software

Master Martini | DSY1104 - Evaluación Parcial 2 | Versión académica de entrega

1. Visión general

Master Martini es una aplicación académica de comercio electrónico para insumos de panadería, pastelería, gelatería y food service. El frontend adapta una referencia visual de Fruitkha de ThemeWagon a un proyecto React. Productos, precios, existencias, cuentas, compras, inscripción en clases y pagos son demostrativos.

2. Alcance y perfiles

Visitante: navegación, categorías, novedades, clases y catálogo. Cliente demo: carrito, checkout simulado, pedidos y clases. Vendedor demo: gestión de productos permitida por la interfaz. Administrador demo: mantenimiento del catálogo y restauración de datos. No se representa un sistema de autenticación seguro para producción.

3. Arquitectura y tecnologías

Frontend: React 19, React Router 8, Vite 7 y Bootstrap 5. Pruebas: Vitest 5, React Testing Library, jsdom y cobertura V8. Estado: ShopProvider/ShopContext; datos de `src/data/catalog.js` y `src/data/clases.js`; utilidades comerciales y persistencia en `src/utils/`. API: servidor Node.js local opcional con proxy en Vite.

4. Flujo de datos y persistencia

La modalidad predeterminada conserva catálogo, carrito, perfil, pedidos e inscripciones mediante localStorage. Al confirmar una compra aprobada se registra el pedido y se descuenta inventario; ante un rechazo se preservan carrito y stock. La API local de demostración dispone de consultas y operaciones sobre productos, pedidos y checkout; puede mantener datos en `server/.data/`.

5. Requisitos funcionales y criterios de aceptación

ID	Requisito	Criterio verificable
RF01	Inicio y navegación	Tres slides y enlaces activos a las vistas.
RF02	Categorías	Acceso a cuatro categorías mediante rutas.
RF03	Catálogo dinámico	Listar productos a partir del estado.
RF04	Búsqueda y filtros	Filtrar por texto, categoría, ofertas y orden.
RF05	Detalle de producto	Mostrar SKU, formato, precio, descripción y stock.
RF06	Ofertas	Mostrar productos con descuento.
RF07	Carrito	Agregar productos respetando el stock.
RF08	Cantidades y total	Modificar, eliminar y calcular importes.
RF09	Registro	Validar RUN, correo, clave, nombre y ubicación.
RF10	Inicio de sesión	Perfiles demostrativos y navegación según rol.
RF11	Checkout	Validar despacho y pago simulado.
RF12	Compra aprobada	Generar pedido y descontar existencias.
RF13	Compra rechazada	No crear pedido ni descontar stock.
RF14	Administración	Acceso visual según rol de demostración.
RF15	CRUD de productos	Crear, consultar, editar y eliminar con validación.
RF16	Historial de pedidos	Ver pedidos de la demostración.
RF17	Novedades	Mostrar artículos y navegación a su detalle.
RF18	Contacto	Validar formulario y mostrar confirmación.
RF19	Responsive	Adaptar navegación, grillas y tarjetas.
RF20	Pruebas de frontend	Suite Vitest de 13 casos; 13/13 aprobados.
RF21	Restauración	Restaurar catálogo de origen.
RF22	Persistencia	Mantener estado compatible entre recargas.
RF23	Integridad de stock	Impedir cantidades superiores al stock o SKU inválido.
RF24	Imagen alternativa	Mostrar recurso de respaldo ante fallos.
RF25	Validación accesible	Usar etiquetas, aria-invalid y mensajes asociados.
RF26	Clases de recetas	Filtrar modalidades; inscribir; mostrar cupos e insumos.

6. Rutas y vistas

Ruta / sección	Componente y objetivo
/	Home.jsx: carrusel, categorías, novedades y ofertas.
/categorias/:category y /catalogo	ShopPages.jsx: productos, filtros, detalles.
/carrito y /checkout	CartCheckout.jsx: importes, validación y resultado de compra.
/login, /registro, /mi-cuenta, /admin	AccountAdmin.jsx: cuentas, pedidos y operaciones administrativas.
/clases	Clases.jsx: recetas, cupos, inscripción e insumos.
/nosotros, /blog, /blog/:slug, /contacto	InfoPages.jsx: información institucional, artículos y contacto.

7. Requisitos no funcionales

Diseño adaptable: componentes responsive para móvil y escritorio. Accesibilidad: campos identificados y estados semánticos. Mantenibilidad: separación entre componentes, datos y utilidades. Rendimiento: assets locales y compilación optimizada mediante Vite. Calidad: pruebas frontend con Vitest y validaciones complementarias de Node/API/medios.

8. Trazabilidad y verificación

Requisitos	Implementación / evidencia
RF01-RF06, RF17-RF19	Home.jsx, ShopPages.jsx, InfoPages.jsx; pruebas de recursos A01-A09.
RF07-RF13, RF21-RF25	ShopContext.jsx, CartCheckout.jsx, ProductMedia.jsx; pruebas de lógica/API.
RF14-RF16	AccountAdmin.jsx; API de demostración y escenarios de integración.
RF20	src/tests/pagina.test.jsx: 13/13 Vitest aprobadas; cobertura V8 medida.
RF26	Clases.jsx y clases.js: validaciones de inscripción, SKU y modalidad.

Resultados cuantitativos

En la ejecución local documentada: 13/13 pruebas Vitest aprobadas, cobertura V8 de 38,40 % de sentencias, 34,83 % de ramas, 37,91 % de funciones y 45,54 % de líneas; compilación Vite exitosa (2.046 módulos, 4,57 s). Se registraron además 39/39 pruebas Node de lógica, 9/9 de imágenes, 15/15 de API y 16/16 comprobaciones smoke. Más información en el informe de testing.

9. Límites y consideraciones

No se procesan pagos reales ni se ofrecen servicios reales de envío o autenticación de producción. Contacto y suscripción son demostrativos. Las capturas E2E conservadas son históricas y no representan necesariamente la interfaz fotográfica actual. Las fotografías de terceros deben respetar sus licencias; la imagen principal `hero-panaderia-artesanal.jpg` conserva marcas de agua de Unsplash+ y requiere comprobar autorización antes de difusión pública.

10. Fuente visual y repositorio

Plantilla de referencia: Fruitkha - Free Bootstrap 4 Responsive Food Business Template (ThemeWagon). Implementación propia adaptada a React y Bootstrap 5. Repositorio: https://github.com/OscarMs1/Master-Martini. Referencia: https://themewagon.com/themes/fruitkha-free-bootstrap-4-responsive-food-business-template/ .